

JavaScript: estruturas de repetição e introdução a arrays

while • do/while • for • contadores • acumuladores • break • continue • arrays

Prof. Dionizio Porfírio
Unichristus • 2026.2

Objetivos de aprendizagem

Ao final da aula, o estudante deverá ser capaz de:

- Utilizar while, do/while e for em problemas de repetição.
- Reconhecer e aplicar contadores e acumuladores.
- Usar break e continue para controlar a execução de um laço.
- Identificar situações em que vários valores devem ser armazenados em um array.
- Criar, acessar, alterar e percorrer arrays utilizando índices.

Elementos de uma estrutura de repetição

A lógica é a mesma já utilizada em Java. O que muda principalmente é a sintaxe da linguagem.

Condição

Expressão avaliada para decidir se uma nova iteração deve ocorrer.

Corpo do laço

Conjunto de comandos executado enquanto a repetição continuar.

Atualização

Mudança de estado que permite ao laço avançar e, em algum momento, terminar.

Cada execução do corpo do laço corresponde a uma iteração.

Estrutura while

A condição é avaliada antes de cada iteração. Se for falsa desde o início, o corpo não é executado.

Sintaxe geral

```
while (condicao) {  
    // comandos repetidos  
}
```

Atenção

Algun valor envolvido na condição deve mudar durante a repetição.

Exemplo: exibir os números de 1 a 5.

```
let numero = 1;  
  
while (numero <= 5) {  
    console.log(numero);  
    numero++;  
}
```

while com quantidade desconhecida

O while é especialmente útil quando não sabemos previamente quantas repetições serão necessárias.

Exemplo Solicitar uma senha até que o usuário informe o valor correto.

```
let senha = "";  
  
while (senha !== "js2026") {  
    senha = prompt("Digite a senha:");  
}  
  
console.log("Acesso liberado");
```

Por que usar while?

A quantidade de tentativas depende das respostas do usuário.

Condição de saída

O laço termina quando a expressão `senha !== "js2026"` passa a ser falsa.

Estrutura do/while

O corpo é executado primeiro; a condição é verificada somente ao final da iteração.

Sintaxe geral

```
do {  
    // comandos repetidos  
} while (condicao);
```

Exemplo

Solicitar um número positivo e repetir a leitura enquanto o valor for inválido.

```
let numero;  
  
do {  
    numero = Number(prompt("Digite um número  
positivo:"));  
} while (numero <= 0);
```

Característica principal

O bloco sempre executa pelo menos uma vez.

Comparação entre while e do/while

while

- Testa a condição antes do corpo.
- Pode executar zero vezes.
- Indicado quando a condição precisa ser válida antes da primeira execução.

Exemplo típico: repetir enquanto ainda houver tentativas disponíveis.

do/while

- Executa o corpo antes do teste.
- Executa pelo menos uma vez.
- Indicado quando uma ação precisa acontecer antes da decisão de continuar.

Exemplo típico: exibir um menu e somente depois verificar a opção escolhida.

Estrutura for

O for concentra a inicialização, a condição e a atualização no cabeçalho da repetição.

```
for (inicializacao; condicao;  
    atualizacao) {  
    // comandos repetidos  
}
```

1. Inicialização

Executada uma única vez, antes do primeiro teste.

2. Condição

Avaliada antes de cada iteração.

3. Atualização

Executada após o corpo da repetição.

Exemplo Exibir os números inteiros de 1 a 5.

```
for (let i = 1; i <= 5; i++) {  
    console.log(i);  
}
```

Variações do for

A atualização não precisa ser apenas `i++`. O contador pode crescer, diminuir ou utilizar outro passo.

Contagem crescente

```
for (let i = 1; i <= 5; i++) {  
    console.log(i);  
}
```

Contagem decrescente

```
for (let i = 5; i >= 1; i--) {  
    console.log(i);  
}
```

Passo diferente de 1

```
for (let i = 0; i <= 10; i += 2) {  
    console.log(i);  
}
```

Regra para o término

A atualização precisa aproximar o contador da condição de saída. Uma atualização incompatível com a condição pode criar um laço infinito.

Contadores

Um contador registra quantas vezes uma situação ocorreu durante a execução do laço.

Exemplo **Ler 5 números e contar quantos deles são positivos.**

Padrão básico

1. Inicialize normalmente com 0.

2. Quando o evento ocorrer, aumente o contador.

```
contador++;  
// ou  
contador += 1;
```

```
let positivos = 0;  
  
for (let i = 1; i <= 5; i++) {  
    let numero = Number(prompt("Digite um número:"));  
  
    if (numero > 0) {  
        positivos++;  
    }  
}  
  
console.log("Positivos:", positivos);
```

Acumuladores

Um acumulador conserva um resultado parcial e incorpora um novo valor a cada iteração.

Padrão para soma

1. Inicialize com 0.

2. Adicione cada novo valor ao total.

```
total = total + valor;  
// ou  
total += valor;
```

Para multiplicações sucessivas, o valor inicial costuma ser 1, e não 0.

Exemplo Ler 4 valores de compra e calcular o total gasto.

```
let total = 0;  
  
for (let i = 1; i <= 4; i++) {  
    let valor = Number(prompt("Valor da compra:"));  
    total += valor;  
}  
  
console.log("Total:", total);
```

Exemplo — resumo de despesas

Solicite ao usuário o valor de 5 despesas. Ao final, apresente:

- o total gasto;
- a média das despesas;
- quantas despesas foram maiores que R\$ 100,00.

Conceitos necessários: for, acumulador, contador, if e conversão da entrada com Number().

Comando break

break encerra imediatamente o laço mais interno em execução.

Exemplo Percorrer os números de 1 a 100 e parar ao encontrar o primeiro múltiplo de 17.

```
for (let numero = 1; numero <= 100;
    numero++) {
    if (numero % 17 === 0) {
        console.log(numero);
        break;
    }
}
```

Efeito

Nenhuma nova iteração é iniciada. A execução continua depois do laço.

Quando usar

Quando uma condição torna desnecessário continuar procurando ou repetindo.

Comando continue

continue encerra apenas a iteração atual e passa para a próxima repetição.

Exemplo Percorrer os números de 1 a 10 e não exibir os múltiplos de 3.

```
for (let numero = 1; numero <= 10; numero++)  
{  
  if (numero % 3 === 0) {  
    continue;  
  }  
  
  console.log(numero);  
}
```

Resultado

3, 6 e 9 são ignorados. Os demais números são exibidos normalmente.

Quando usar

Quando alguns casos devem ser descartados sem interromper o restante do laço.

Diferença entre break e continue

break

Encerra o laço inteiro.

Depois do break, nenhuma nova iteração daquele laço é executada.

Uso típico: interromper uma busca assim que o resultado desejado for encontrado.

continue

Encerra somente a iteração atual.

O laço continua normalmente a partir da próxima iteração.

Uso típico: ignorar entradas inválidas ou casos que não devem ser processados.

Exemplo — controle de tentativas

Um sistema permite no máximo 5 tentativas para informar o código "JS2026".

- A cada tentativa, solicite o código pelo prompt.
- Se a entrada estiver vazia, ignore essa passagem com continue.
- Se o código estiver correto, encerre imediatamente o laço com break.
- Ao final, informe se o acesso foi liberado ou bloqueado.

O exercício combina limite de tentativas, condição, continue, break e uma variável lógica para registrar o resultado.

Arrays em JavaScript

Quando vários valores pertencem ao mesmo conjunto, um array permite armazená-los sob um único identificador.

Variáveis separadas

```
let nota1 = 7.5;  
let nota2 = 8.0;  
let nota3 = 6.5;  
let nota4 = 9.0;
```



Array

```
let notas = [7.5, 8.0, 6.5, 9.0];
```

Relação com Java

O conceito é semelhante ao vetor: os valores são acessados por posição. Em JavaScript, a estrutura é mais dinâmica.

Criação de arrays

Há mais de uma forma de criar um array. Na maioria dos casos, a notação literal com colchetes é a forma mais direta.

Array vazio

```
let nomes = [];
```

Array já inicializado

```
let notas = [7, 8.5,  
10];
```

Construtor Array

```
let cidades = new  
Array("Fortaleza",  
"Quixadá");
```

Tamanho inicial

```
let valores = new  
Array(5);
```

Atenção

`new Array(5)` cria um array com comprimento 5 e posições vazias; não cria o array `[5]`. Para os exemplos iniciais da disciplina, prefira a forma literal com `[]`.

Índices e propriedade length

Os índices começam em 0. Portanto, um array com 4 elementos possui índices de 0 a 3.

```
let notas = [7.5, 8.0, 6.5, 9.0];  
  
console.log(notas[0]);           // 7.5  
console.log(notas[2]);           // 6.5  
console.log(notas.length);       // 4  
console.log(notas[notas.length - 1]); // 9.0
```

Índice

Representa uma posição dentro do array.

length

Representa a quantidade de elementos. Por isso, o último índice válido é sempre `length - 1`.

Leitura e alteração de elementos

Exemplo Alterar a segunda nota e adicionar um novo valor na próxima posição disponível.

```
let notas = [7.5, 8.0, 6.5];  
  
notas[1] = 9.0;  
notas[notas.length] = 10.0;  
  
console.log(notas);
```

Alteração

Atribuir um valor a `array[indice]` substitui o conteúdo daquela posição.

Próxima posição

Como o último índice é `length - 1`, o índice `length` representa a próxima posição livre ao final do array.

Percorrendo um array com for

Exemplo Exibir todas as notas armazenadas e suas respectivas posições.

```
let notas = [7.5, 8.0, 6.5, 9.0];

for (let i = 0; i < notas.length; i++) {
  console.log("Índice:", i);
  console.log("Nota:", notas[i]);
}
Ou

for (const nota of notas) {
  console.log(nota);
}
```

Valor inicial

`i` começa em 0 porque o primeiro elemento do array está no índice 0.

Condição

`i < notas.length` garante que o laço pare antes de acessar uma posição inexistente.

Exemplo — notas da turma

Leia 5 notas pelo prompt e armazene todas em um array.

- Primeiro, utilize um laço para preencher o array.
- Depois, percorra novamente o array para calcular a média da turma.
- Conte quantas notas são maiores ou iguais a 6.
- Exiba todas as notas registradas antes de apresentar o resultado final.

A separação entre preenchimento e processamento permite reutilizar os valores depois que a leitura terminou.

Erros frequentes nesta aula

Laço sem atualização

A condição permanece verdadeira porque a variável de controle não é modificada.

Condição incompatível

O contador avança em uma direção que nunca alcança a condição de término.

Entrada sem conversão

prompt() retorna string. Use Number() quando o valor será utilizado em cálculos.

Índice fora do array

Com length elementos, os índices válidos vão de 0 até length - 1.

Exercício 1 — processamento de vendas

Uma loja deseja analisar uma sequência de vendas durante o expediente.

- O programa deve permitir o registro de até 10 valores de venda.
- Se o usuário informar 0, o registro deve ser encerrado imediatamente.
- Valores negativos são inválidos e devem ser ignorados, sem entrar nos cálculos.
- Ao final, mostre a quantidade de vendas válidas, o total vendido e a média das vendas válidas.
- Caso nenhuma venda válida tenha sido registrada, não calcule a média.

Exercício 2 — análise de notas

Crie um programa para registrar e analisar as notas de 6 estudantes.

- Leia cada nota pelo prompt e aceite somente valores entre 0 e 10. Enquanto uma nota for inválida, solicite-a novamente.
- Armazene as 6 notas em um array.
- Depois do preenchimento, percorra o array e calcule a média da turma.
- Informe a maior nota registrada e quantos estudantes obtiveram nota maior ou igual a 6.